

CSC 110L

Lab 4: Booleans, Selection, and Iteration

Due Date: Thursday, September 25, 11:59pm

Learning Goals

- Practice writing and modifying code that uses Boolean expressions
- Practice writing and modifying code that uses the `if`, `else`, and `while` structures

Instructions

Part A

1. Without using JShell or any program, predict what the result of each of the following Boolean expressions would be, if they were used in a Java program. Assume the variables that I'm using in the table have been initialized as shown here:

```
boolean x = true;
boolean y = false;
boolean z = (x || y);
int num = 7;
int count = -1;
```

Your grade is not based on the correctness of your answer. As long as it is a reasonable prediction, you will receive credit. The purpose of this exercise is to get you to think about Java handles expressions like these without having to enter them. (5 points)

Expressions	Predicted Output
<code>(x && false)</code>	false
<code>(true y)</code>	true

<code>(num == 7)</code>	true
<code>((x y) && (count > 0))</code>	false
<code>(true && true && num == 7 && false)</code>	false
<code>(!y (count >= 0) (num != 7))</code>	true
<code>!(y (count >= 0) (num != 7))</code>	true
<code>(!(x && (count < 0))) && z</code>	false

2. Use JShell to see the actual output of the same expressions. Give the output in the middle column below. Also make a note about whether your prediction was correct or not. Do not go back and change your predictions.

For any predictions that you got wrong, use the space below the table to explain what you misunderstood. If you're not sure, ask the professor! (4 points)

Expressions	JShell Output	Prediction correct?
<code>(x && false)</code>	false	correct
<code>(true y)</code>	true	correct
<code>(num == 7)</code>	true	correct
<code>((x y) && (count > 0))</code>	false	correct
<code>(true && true && num == 7 && false)</code>	false	correct
<code>(!y (count >= 0) (num != 7))</code>	true	correct
<code>!(y (count >= 0) (num != 7))</code>	true	correct
<code>(!(x && (count < 0))) && z</code>	false	correct

Explain any predictions that you got wrong here. Why did you get them wrong? What did you misunderstand? (Ask the professor if you need help!)

I didn't get any wrong, surprisingly. I had a question on the last one where I wanted to know where ! was being used towards. I already had good intuition and all of it seems simple

3. For each program below, state how many times the word “Quack” will be printed. Answer these questions without running the code in JShell or in a Java program. Work through the problem by hand. (4 points)

<pre>// 3a void main(){ int count = 0; while(count <= 10){ IO.println("Quack"); count = count + 1; } }</pre> // How many times does Quack print? 11 quacks. The first column is the index, while the second is the index. <pre>0 1 quack 1 2 quack 2 3 quack 3 4 quack 4 5 quack 5 6 quack 6 7 quack 7 8 quack 8 9 quack 9 10 quack 10 11 quack</pre>	<pre>// 3b void main(){ int count = 0; while(count > 0){ IO.println("Quack"); count = count + 1; } }</pre> // How many times does Quack print? None. $0 > 0$ is false.
<pre>// 3c void main(){ int count = 0; while(count >= 0) { IO.println("Quack"); count = count + 1; } }</pre> // How many times does Quack print? Infinite. Count++ without a line dedicated to preventing an infinite loop	<pre>// 3d void main(){ int count = 0; while(count < 15){ if(count % 3 == 0) { IO.println("Quack"); } else { IO.println("Moo"); } count = count + 1; } }</pre> // How many times does Quack print? 4 quacks $3 \% 3 = 0$, 1 Quack

	<pre> 6 % 3 = 0, 2 Quacks 9 % 3 = 0, 3 Quacks 12 % 3 = 0, 4 Quacks 15 % 3 (statement is count < 15) it would stop at 14 with "moo" </pre>
--	--

Part B

- BobcatDetector.java is meant to be a program that determines whether the user is a bobcat. Currently, it asks the user if they like hockey. (Bobcats like hockey, but so do some other creatures.) If the user likes hockey, the program currently prints that the user might be a bobcat. Otherwise, the program states that the user is not a bobcat.

Modify the program so that if the user likes hockey, the program asks a second question: Do you like Spuds baked potatoes? If the user says yes, then the program should say that they are definitely a bobcat. Otherwise, the program should say that they are not a bobcat. *(6 points)*

- If you run BrokenLoop.java, it currently results in an infinite loop. (You can try running it – don't forget that you can use Ctrl+C to cancel a running program.) Fix BrokenLoop.java so that the loop runs exactly 8 times. You may only modify the while loop condition to make this change. *(3 points)*

Part C

- Write a program called PositiveChecker.java that repeatedly asks the user to enter an integer. If the integer is positive, the program should say "That is a positive number." If the integer is negative, the program should say "That is a negative number." If the integer is zero (which is neither positive or negative), then the program should say "Program ended." and end. *(9 points)*

Here's an example of how the program should behave (with user input in red):

```

Enter a number: 5
That is a positive number.
Enter a number: -3
That is a negative number.
Enter a number: 0

```

Program ended.

Submitting Your Code

When you are done, create **one** zip file that contains this file with your answers to Part A, and all of your .java files from this lab (BobcatDetector.java, BrokenLoop.java, and PositiveChecker.java). Do not rename any of the files. Submit the .zip file on Blackboard by the due date.